Ders 5 Genişletilmiş Notu: Prefix Kodlar, Unary ve Golomb Kodları, Shannon-Fano Kodlama

Ders 5 Genişletilmiş Notu: Prefix Kodlar, Unary ve Golomb Kodları, Shannon-Fano Kodlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. İstatistiksel Yöntemlere Giriş
- 2. Sabit ve Değişken Uzunluklu Kodlar
- 3. Prefix Olan ve Olmayan Kod: Bir Örnek
- 4. Prefix Kodun Tanımı ve Kuralları
- 5. Hazır Prefix Kod: Başa Sıfır Eklemek
- 6. Unary Kod ve Start-Step-Stop
- 7. C1, C2, C3 ve C4 Prefix Kodları
- 8. Golomb Kodu
- 9. Hazır Kodlardan Yöntem Ürünü Kodlara
- 10. Shannon-Fano Kodlama
- 11. Shannon-Fano Örneği, Kod Çözme ve Değerlendirme
- 12. İkinci Shannon-Fano Örneği
- Örnek Sorular

1. İstatistiksel Yöntemlere Giriş

Hoca derse önceki haftanın sonunda kalınan yerden, prefix kodlardan başladı. Slayttaki başlık "İstatistiksel Yöntemler"dir ve altında şu konular sıralanır (ekrandan, 00:10:00):

- Prefix Kodlar
- Shannon Fano Coding
- Huffman Coding
- Arithmetic Coding
- Diğer Yöntemler

Hoca sıkıştırma yöntemlerinin istatistiksel yöntemler, sözlüksel yöntemler ve diğer yöntemler diye ayrıldığını hatırlattı. Diğer yöntemler başlığı altında faks sıkıştırmada kullanılan yöntemleri ve PPM'i saydı. PPM de istatistiğe dayalıdır, ama karakterleri tekli ya da ikili değil, kombinasyonlar hâlinde ele alır.

Veriyi hangi yöntemle sıkıştırırsak sıkıştıralım iki şey önemlidir. Birincisi, sıkıştırılan dosya geri açılabilmelidir; açılamıyorsa sıkıştırmanın "bir esprisi yoktur". İkincisi, istatistiksel yöntemlerle çalışılıyorsa dosyanın geri açılabilmesi için sembollere (ya da piksellere) verilen kodların **prefix kod** özelliğine sahip olması gerekir.

2. Sabit ve Değişken Uzunluklu Kodlar

Kodlar ya sabit uzunlukludur ya da değişken uzunluklu. Bilgisayarda kullanılan ASCII kodu sabit uzunlukludur: her karakter 8 bittir. Sabit uzunluklu kodlar sıkıştırma performansı sağlamaz, çünkü veride çok fazla artıklık (redundancy) kalır; bazı şeyler defalarca söyleniyor gibi-

dir ve veride hâlâ sıkıştırılabilecek bir şey vardır. Bu yüzden değişken uzunluklu kodlar seçilir. Prefix kodlar da değişken uzunluklu kodlardandır, ama bir kodun prefix diye adlandırılması için özel bir koşul vardır.

2.1 Eşit olasılıklı dört sembol

Hoca konuyu entropiyle bağladı (ekrandan, 00:20:00). Alfabede dört sembol olsun: $\{a_1, a_2, a_3, a_4\}$, hepsinin olasılığı $P_i = 0{,}25$. Her sembolün olasılığını bilmek, o sistemin kaynağını bilmek demektir.

Entropi, kaynağı kodlamak için gereken ortalama bit uzunluğudur. Bir sembolün entropiye katkısı olasılığı çarpı taşıdığı bilgidir; bilgi $-\log_2 P_i$ ile ölçülebilir hâle getirilir. n olay için:

$$H = -\sum_{i=1}^{n} P_i \log_2 P_i$$

Dört eşit olasılıklı sembol için terimi dört kez yazmak yerine dörtle çarparız:

$$H = -(4 \cdot 0{,}25 \log_2 0{,}25) = 2 \text{ bit}$$

Entropiden haberi olmayan biri de bu dört sembolü birbiriyle çakışmayacak şekilde kodlamak isterse 2 bitle $2^2 = 4$ farklı kod yazar: `00`, `01`, `10`, `11`. Bunlar sırasıyla a_1, a_2, a_3, a_4'ün kodu olur. Ortalama mesaj uzunluğu S: toplam $2 + 2 + 2 + 2 = 8$ bit, dörde bölünce $S = 2$ bit. Artıklık $R = |S - H| = 0$. Bu veri 2 bitin altında kodlanamaz.

Ama gerçek dünya böyle değildir. Bir resimdeki piksellerin hepsi aynı sayıda değildir; bir dokümandaki bütün karakterlerin kullanım sıklığı aynı değildir. Resmi ortadan bölüp yarısını siyah yarısını beyaz yaparsak eşit olur, ama bu özellikle hazırlanmış bir durumdur. Artıklığın sıfır olması yalnız teoride mümkündür.

2.2 Farklı olasılıklı dört sembol

Aynı dört sembolün olasılıkları $P(a_1) = 0{,}49$, $P(a_2) = 0{,}25$, $P(a_3) = 0{,}25$, $P(a_4) = 0{,}01$ olsun (ekrandan, 00:20:00):

$$H = -(0{,}49 \log_2 0{,}49 + 0{,}25 \log_2 0{,}25 + 0{,}25 \log_2 0{,}25 + 0{,}01 \log_2 0{,}01) = 1{,}57 \text{ bit}$$

Formülün başındaki eksi, $I = \log_2(1/P)$ yazımındaki P'nin paydan yukarı çıkarken aldığı eksiden gelir. Teori bu olasılıklara sahip bir doküman ya da resim için ortalama 1,57 bit gerektiğini söyler. Ama 1,57 bit gibi gerçel bir değer dünyamızda yoktur; bit sayısı tam sayı olmalıdır. Demek ki sembollerin bazıları 1, bazıları 2, bazıları 3 bitle kodlanacak ve ortalama 1,57'ye yaklaşılacaktır. Bunu yapmanın yolları Huffman, aritmetik kodlama gibi yöntemlerdir; bu yöntemlerin sonucu da prefix kodlardır. Bir de ASCII tablosu gibi önceden hazırlanmış, değişken uzunluklu hazır prefix kodlar vardır.

Olasılıklara hiç bakmadan bu dört sembolü yine 2'şer bitle kodlarsak:

$$R = |S - H| = |2 - 1{,}57| = 0{,}43$$

Önceki örnekte artıklık sıfırdı, çünkü olasılıklar eşitti. Burada olasılıklar farklıyken sabit uzunluklu kod kullanmak artıklığı büyüttü. Demek ki sabit uzunluklu değil, değişken uzunluklu kod kullanmak gerekir.

3. Prefix Olan ve Olmayan Kod: Bir Örnek

3.1 Girdi ve iki kod

Slayttaki girdi şudur (ekrandan, 00:30:00):

`a1 a3 a2 a1 a3 a3 a4 a2 a1 a1 a2 a2 a1 a1 a3 a1 a1 a2 a3 a1`

Hoca sembolleri saydı: 9 tane a_1, 5 tane a_2, 5 tane a_3 ve 1 tane a_4; toplam 20 sembol. Olasılıklar aynı çıkmaz. Henüz bir yöntem bilmediğimizi varsayarak bu sembollere iki farklı kod atanmıştır:

Sembol	Olasılık (slayt)	Code1	Code2
a_1	0,49	`1`	`1`
a_2	0,25	`01`	`01`
a_3	0,25	`010`	`000`
a_4	0,02	`001`	`001`

En sık kullanılan a_1 en kısa kodu (1 bit) alır. Bilgi teorisine göre bir sembolün karşımıza çıkma olasılığı ne kadar yüksekse taşıdığı bilgi o kadar azdır; bilginin azlığı daha az bitle ifade edilmesine karşılık gelir. Olasılığı düşük olan sembol gelirse taşıdığı bilgi büyüktür ve daha fazla bitle ifade edilir. Hoca a_2 ile a_3'ün olasılıkları aynı olduğu hâlde birine 2, diğerine 3 bit vermesini "öyle attım" diye açıkladı; bunun neden olamayacağını ileride göstereceğini söyledi. Code2'yi "başka bir arkadaşımız" vermiş olsun: ilk iki kod aynı, üçüncü farklı, dördüncü aynı.

3.2 Kodlama ve kod çözme

Soru şudur: Bu iki kodun ikisiyle de sıkıştırma yapılabilir mi? Girdi `a1 a3 a1` olsun.

- Code1 ile: `1` + `010` + `1` = `10101` (5 bit).
- Code2 ile: `1` + `000` + `1` = `10001` (5 bit).

İkisi de 5 bit; her şey yolunda görünür. Şimdi açalım.

Code1 ile çözme. İlk bit `1`; 1 olan bir kod var ve 1 ile başlayan başka kod yok, bu a_1'dir. Sonra `0` gelir; tek bitlik `0` kodu yoktur. İki bit alınır: `01`. Bu a_2'nin kodudur. Ama bir bit daha alınırsa `010` olur, bu da a_3'ün kodudur. Şimdi `01`'i alıp a_2 mi demeli, yoksa üç biti alıp a_3 mü? Bilinmez. a_2 alınıp da devam edilebilir, a_3 alınıp da. Böyle bir ikilemde bazen metnin sonuna kadar gidilip her şey çözülür ama bambaşka bir içerik çıkar; bazen de öyle bir yerde kalınır ki ilerlenemez, eşleşen bir kod yoktur.

Code2 ile çözme. 1 geldi; 1 ile başlayan başka kod olmadığı için a_1. Sonra `0`; 0 ile başlayan birçok kod var. İkinci bit: `00`; bununla başlayan iki kod var (`000` ve `001`). Üçüncü bit: `000`; yalnız a_3 böyledir. Sonra $1 \to a_1$. Hiçbir problem yaşanmadı.

Bunun sebebi şudur: **Code1 prefix kod değildir, Code2 prefix koddur.** Bir kod sistemiyle kodlanan dokümanı geri açamıyorsak o kodun hiçbir esprisi yoktur. Kodlanan veri geri açılabiliyorsa bu doğru kod demektir ve prefix özelliğine sahiptir.

3.3 Bit sayısı ve artıklık

Code1 ile kodlandığında (ekrandan, 00:30:00):

- 9 tane $a_1 \to 1$ bit $\to$ 9 bit
- 5 tane $a_2 \to 2$ bit $\to$ 10 bit
- 5 tane $a_3 \to 3$ bit $\to$ 15 bit
- 1 tane $a_4 \to 3$ bit $\to$ 3 bit

Toplam $9 + 10 + 15 + 3 = 37$ bit. Hoca önce slayttaki "$a_4 \to 1$ bit" satırıyla toplayıp 37'yi tutturamadı, sonra a_4'ün kodunun 3 bit olduğunu söyleyip slaytı düzeltti. Code1 ve Code2'deki bit uzunlukları aynı olduğu için ikisiyle de 37 bit çıkar. ASCII ile 20 sembol $20 \cdot 8 = 160$ bit tutardı; 160 bit yerine 37 bit kullanılmış olur.

$$S = \frac{37}{20} = 1{,}85 \text{ bit}, \qquad H = 1{,}57, \qquad R = |H - S| = 0{,}28$$

Hoca 0,28'in sıfıra yakın, iyi bir sıkıştırma performansı olduğunu söyledi. Ancak Code1'in geri dönüşü yoktur, Code2'nin vardır; nedeni prefix özelliğidir.

> **Ders sonrası not:** Slayttaki olasılıklar ($0{,}49$; $0{,}25$; $0{,}25$; $0{,}02$; toplamı $1{,}01$) ve $H = 1{,}57$ önceki örnekten alınmıştır. Girdideki sayımlara göre olasılıklar $9/20 = 0{,}45$, $5/20 = 0{,}25$, $5/20 = 0{,}25$, $1/20 = 0{,}05$'tir. Bu dağılımın entropisi python ile yaklaşık $1{,}73$ bit çıkar; buna göre artıklık $|1{,}85 - 1{,}73| \approx 0{,}11$ bittir. Hoca 00:21:19'da olasılıkları sayarken 9/20 ve 1/20 değerlerini söyledi, sonra slayttaki değerlere geçti.

4. Prefix Kodun Tanımı ve Kuralları

4.1 Tanım

Bir sembolün sahip olduğu kodun bit deseni (bit pattern) diğer bir sembolün kodunun ön eki olamaz. Code1'de kontrol edelim: `1` ile başlayan başka kod yok, doğru. `01` ile başlayan başka kod olmamalı, ama `010` var. İşte problem budur; açarken iki biti alıp a_2 mi, üçüncü bite bakıp a_3 mü diyeceğimize karar veremeyiz. Code2'de: `1` ile başlayan başka kod yok, `01` ile başlayan yok, `000` ile başlayan yok, `001` ile başlayan yok. Öyleyse Code2 prefix koddur.

Bütün istatistiksel yöntemlerin önerdiği, yani yöntem sonucunda elde edilen kodların hepsi prefix kod olmalıdır. Hoca bunu veri sıkıştırmanın "olmazsa olmazlarından biri" diye niteledi.

4.2 Değişken uzunluklu kodun iki kuralı

Değişken uzunluklu kod oluşturulurken iki temel özelliğe dikkat edilir:

1. En sık kullanılan sembole en kısa kod atanır. Sıklığı fazla olanın olasılığı büyüktür; olasılık ile bilgi ters orantılı olduğu için taşıdığı bilgi azdır ve daha az bitle ifade edilir.
2. Kod prefix özelliğine uymalıdır: bir sembolün kodu başka bir sembolün kodunun başlangıcını oluşturamaz.

Adın kaynağı İngilizcedeki eklerdir: prefix öne gelen ek, suffix sona gelen ek, infix ortaya gelen ektir. Prefix kodda bir kod başka bir kodun ön eki olamaz.

4.3 Soru: Tam sayıların binary gösterilimi prefix midir?

Hoca şu soruyu sordu: Integer sayıların binary gösterilimi prefix kod özelliğine uyar mı? 0'dan 8'e kadar sayıların binary karşılıklarını yazdı: `0`, `1`, `10`, `11`, `100`, `101`, `110`, `111`, `1000`. Sonra kontrol etti: `1` ile başlayan birçok kod var (`10`, `11`, `100`, …); `10` ile başlayan kodlar var (`100`, `101`, `1000`); `100` ile başlayan kod var (`1000`). Bu böyle devam eder. Öyleyse tam sayıların binary gösterilimi prefix kod özelliğine uymaz ve prefix kod olarak kullanılamaz.

Bir öğrencinin aklına şu gelebilir: En büyük sayı 8 ve 4 bitle yazılıyor; hepsini 4 bitle yazsak prefix olur. Hoca bunun doğru olduğunu, ama o zaman kodun değişken uzunluklu değil sabit uzunluklu olacağını söyledi. Sabit uzunluklu kodlarda artıklığın yüksek çıkacağı bilinir; amaç onu düşürmektir.

5. Hazır Prefix Kod: Başa Sıfır Eklemek

5.1 Hazır prefix kodlar

Yöntemlere geçmeden önce araştırmacıların belirli kurallarla ürettiği ve kullanıma sunduğu hazır prefix kodlar vardır. Burada "sembol" karakter, görüntü verisi ya da ses verisinden çıkarılan bir veri olabilir; sembol her şey olabilir. Farklı olasılıklarla gelen sembolleri kodlamak için farklı uzunluklarda prefix kodlar hazırlanır. Ancak böyle hazır kodlarda, veriye bağlı olarak istatistiksel bir yöntemle kod çıkarılmadığı için artıklık ister istemez biraz büyük olacaktır.

5.2 Alt ve üst tam değer

Kuralı okumak için iki işaret gerekir (ekrandan, 00:40:00 ve 00:44:40). Hoca bu işaretleri ilk kez lisede gördüğünü, kitabın köşeli parantezi yanlış bastığını sandığını anlattı. Sayı doğrusunda 7 ile 8 arasında 7,2'yi düşünelim:

- $\lceil 7{,}2 \rceil$: 7,2'den büyük ya da eşit en küçük tam sayı, yani 8.
- $\lfloor 7{,}2 \rfloor$: 7,2'den küçük ya da eşit en büyük tam sayı, yani 7.

Bir küsurlu bir değerin alt tam değeri 1'dir.

5.3 Kural ve kodlar

Kural şudur (ekrandan, 00:44:40): $i \geq 1$ olmak şartıyla, i. kodun binary değerinin başlangıcına $\lfloor \log_2 i \rfloor$ adet sıfır eklenir.

i	kod	uzunluk
1	`1`	1
2	`010`	3
3	`011`	3
4	`00100`	5
5	`00101`	5
6	`00110`	5
7	`00111`	5
8	`0001000`	7
…	…	…
15	`0001111`	7
16	`000010000`	9

Hocanın çözdüğü örnekler:

- $i = 1$: binary `1`; $\log_2 1 = 0$; başa 0 sıfır eklenir → `1`.
- $i = 2$: binary `10`; $\log_2 2 = 1$; başa bir sıfır → `010`.
- $i = 5$: binary `101`; $\log_2 4 = 2$ olduğuna göre $\log_2 5$ 2'den büyük bir sayıdır; alt tam değeri 2; başa iki sıfır → `00101`.
- $i = 15$: binary `1111`; $\log_2 16 = 4$ olduğuna göre $\log_2 15$ 3 küsurdur (yaklaşık 3,9); alt tam değeri 3; başa üç sıfır → `0001111`.

Kontrol: `1` ile başlayan başka kod yok; `010` ile başlayan yok; `011` ile başlayan yok; `00100` ile başlayan yok. Tablodaki kodların hepsi prefix koddur. Uzunluklar 1, 3, 3, 5, 5, 5, 5, 7, …, 9 diye gider. Olasılık değerleri büyükten küçüğe sıralanır ve kodlar bu sırayla verilir; en olası sembol en kısa kodu alır.

> **Ek bilgi (derste anlatılmadı):** Bu kod literatürde büyük olasılıkla Elias gamma kodu olarak bilinir. Başa eklenen sıfırların sayısı, arkadan gelen binary sayının kaç bit olduğunu söyler; çözücü sıfırları sayıp o kadar bit daha okur.

5.4 Kod uzunluğu formülü

Hoca bu tür hazır kodlarda içerik bilinmeden kullanılırsa artıklığın yüksek olacağını tekrarladı: Elimizde 0,49 olasılıklı iki veri olsun; birini 1 bitle, aynı olasılıktaki diğerini 3 bitle kodlarsak ister istemez artıklık olur.

Tabloyu oluşturmadan bir kodun uzunluğunu bulmak için formül vardır (ekrandan, 00:47:20):

$$\text{uzunluk}(i) = 1 + 2\lfloor \log_2 i \rfloor \text{ bit}$$

- $i = 16$: $16 = 2^4$, $\log_2 16 = 4$; $2 \cdot 4 = 8$; $8 + 1 = 9$ bit.
- $i = 6$: $\log_2 4 = 2$ olduğuna göre $\log_2 6$ 2'den büyüktür; alt tam değeri 2; $2 \cdot 2 + 1 = 5$ bit.

Böylece "64. sıradaki kodun uzunluğu nedir?" diye sorulduğunda kodları tek tek çıkarmadan formülle bulunur.

Ders sonrası not: Hoca 00:46:37'de 16 için "eşit ve kendisinden büyük integer sayı 4" dedi; formüldeki işaret alt tam değerdir (kendisinden küçük ya da eşit en büyük tam sayı). $\log_2 16 = 4$ tam sayı olduğu için sonuç değişmez.

6. Unary Kod ve Start-Step-Stop

6.1 Tanım

Unary kod ilerleyen kodlarda da kullanılacak önemli bir koddur. Slayttaki tanım (ekrandan, 01:10:00): Negatif olmayan n integer sayısının unary kodu $(n-1)$ adet 1 ve onları takip eden bir tane 0'dan oluşur; ya da tam tersi, $(n-1)$ tane 0 ve onları takip eden bir tane 1'den oluşur.

n bazen 1'den, bazen 0'dan başlatılır:

- n 1'den başlıyorsa: $(n-1)$ adet 1 ve bir 0.
- n 0'dan başlıyorsa: $0-1$ adet 1 yazılamayacağı için n adet 1 ve bir 0.

n	code	alter code
1	`0`	`1`
2	`10`	`01`
3	`110`	`001`
4	`1110`	`0001`
5	`11110`	`00001`

$n=1$ için $1-1=0$ tane 1 yazılır, ardından bir 0 gelir: `0`. $n=2$ için bir tane 1 ve 0: `10`. $n=3$ için iki tane 1 ve 0: `110`. $n=4$ için üç tane 1 ve 0: `1110`. Alternatif kod, sıfırların yerine bir, birlerin yerine sıfır konarak elde edilir. Hoca örneklerinde genelde "$(n-1)$ adet 1, onu takip eden 0" biçimini kullandığını söyledi.

6.2 Start-step-stop tripleti

Unary kodlar "start-step-stop" kodu olarak da bilinir. Triplet bir üçlüdür: start (başlangıç), step (adım), stop (bitiş noktası). Kodları çıkarmak için bir a değişkeni hesaplanır:

$$a = \text{start} + n \cdot \text{step}$$

$a = \text{stop}$ olduğunda a bitinin önündeki 0 düşürülür.

Bunun amacı şudur: "1750 tane unary kod yaz" denirse her biri için "$n-1$ tane 1, arkasına 0" işlemini yapmak çok vakit alır. Start, step ve stop üçlüsüyle bu kolayca formülize edilir. Üçlü her zaman tam istenen sayıyı vermeyebilir, daha fazla kod sayısını ifade edebilir; o zaman yalnız gereken kadarı oluşturulur.

6.3 Örnek: 680 kod

Slayttaki örnek: 680 adet unary kod nasıl elde edilir? 680 kod için triplet $(3, 2, 9)$'dur (ekrandan, 01:10:00). Start 3, step 2, stop 9.

n	$a = 3 + 2n$	n. kod sözcüğü	kod sözcüğü sayısı	integer sınır
0	3	`0xxx`	$2^3 = 8$	0-7
1	5	`10xxxxx`	$2^5 = 32$	8-39
2	7	`110xxxxxxx`	$2^7 = 128$	40-167
3	9	`111xxxxxxxxx`	$2^9 = 512$	168-679

Adım adım: $n=0$ için $0 \cdot 2 + 3 = 3$; $n=1$ için $1 \cdot 2 + 3 = 5$; $n=2$ için $2 \cdot 2 + 3 = 7$; $n=3$ için $3 \cdot 2 + 3 = 9$.

Unary kısım: n burada 0'dan başladığı için n adet 1 ve bir 0 yazılır: $n = 0 \rightarrow$ `0`, $n = 1 \rightarrow$ `10`, $n = 2 \rightarrow$ `110`, $n = 3 \rightarrow$ `1110`. Ama $n = 3$'te $a = 9 =$ stop olduğu için sondaki 0 düşürülür ve `111` kalır.

Arkadaki x'lerin sayısı a kadardır: 3 tane x ile $2^3 = 8$ kod sözcüğü yazılır (0-7), 5 tane x ile 32 (8-39), 7 ile 128 (40-167), 9 ile 512 (168-679). Yani önce 3 bitlik `000`, `001`, …, `111`; sonra 5 bitlik sayılar; sonra 7 bitlik; en son 9 bitlik sayılar yazılır. Bu binary sayılar tek başına prefix değildir; önlerine unary kodları eklenince otomatik olarak prefix olurlar.

Sıfırın neden düşürüldüğü: `0` ile başlayan grup, `10` ile başlayan grup, `110` ile başlayan grup birbirinden ayrılır. Son gruptan sonra başka grup gelmeyecektir; `111` ile başlayıp `1110` ile devam eden ayrı bir grup olmayacağı için o 0 gereksizdir. Eğer $n = 4$ diye bir grup daha olsaydı bu 0'ın kalması gerekirdi, çünkü ileride üç tane 1 ile başlayan başka bir kod çıkabilirdi.

Sonuç: 0 dahil 679'a kadar 680 tane prefix kod kolayca çıkarılmış olur.

6.4 Kod sayısı formülü

Triplet verildiğinde kaç kod oluşturulacağı şu formülle bulunur (ekrandan, 01:14:40):

$$\frac{2^{\text{stop}+\text{step}} - 2^{\text{start}}}{2^{\text{step}} - 1} = \frac{2^{11} - 2^3}{2^2 - 1} = \frac{2048 - 8}{3} = 680$$

Kod sayısı stop parametresine bağlıdır: a stop değerine geldiğinde durulur, sonrası ilgilendirmez. Bu yüzden sayı stop'a bağlı olarak **üstel** artar. Stop 3 olsaydı 8 kod; 5 olsaydı $8+32 = 40$; 7 olsaydı $40 + 128 = 168$; 9 olunca 680 kod elde edilir. Stop 11 olsaydı sayı daha da çok artacaktı.

> **Ders sonrası not:** Hoca 01:16:13'te stop 7 için önce "160 tane", sonra "167 tane" dedi; $8 + 32 + 128 = 168$ koddur (0-167 aralığı).

6.5 Hazır kodlarda artıklık

Hoca bu tablonun ilk grubu üzerinden bir uyarı yaptı: İlk 8 sembolün hepsi aynı uzunlukta (4 bit, `0xxx`) kodlanır, olasılıkları ne olursa olsun. Olasılıklar 0,6; 0,4; 0,3 gibi hızla değişiyorsa ve hepsi aynı bit uzunluğunda kodlanıyorsa artıklık yüksek olur. Olasılıklar 0,65; 0,64; 0,63; 0,62 gibi birbirine çok yakınsa aynı bit uzunluğu sorun yaratmaz, artıklık küçük olur. Yani hazır kod veriye bağlı olarak çok iyi de çok kötü de sonuç verebilir.

7. C1, C2, C3 ve C4 Prefix Kodları

7.1 Gösterimler

Hoca dört prefix kod daha gösterdi: C_1, C_2, C_3, C_4 (ekrandan, 01:20:00). Önce üç gösterim tanımlanır (n integer sayılar: 1, 2, 3, …):

- $B(n)$: n sayısının binary karşılığı (1 → 1, 2 → `10`, 3 → `11`, 4 → `100`).
- $|B(n)|$: binary sayının dijit uzunluğu. Kümelerde mutlak değer işaretinin eleman sayısını (kardinaliteyi) göstermesi gibidir (1 → 1, 2 → 2, 3 → 2, 4 → 3).
- $\bar{B}(n)$: $B(n)$'nin en soldaki biti atıldığında kalan bitler. En soldaki bit her zaman 1'dir (1 → boş, 2 → `0`, 3 → `1`, 4 → `00`, 5 → `01`, 6 → `10`).

Slayttaki tablonun bir kısmı (ekrandan, 01:20:00). Slaytta kodların bölümleri dikey çizgiyle ayrılır; aşağıdaki tabloda bölümler boşlukla ayrılmıştır:

n	$B(n)$	$\bar{B}(n)$	C_1	C_2	C_3	C_4
1	1		0	0	0	0
2	10	0	10 0	100	100 0	10 0
3	11	1	10 1	110	100 1	11 0
4	100	00	110 00	10100	110 00	10 100 0
8	1000	000	1110 000	1010100	10100 000	11 1000 0

n	$B(n)$	$\bar{B}(n)$	C_1	C_2	C_3	C_4
15	1111	111	1110 111	1111110	10100 111	11 1111 0
16	10000	0000	11110 0000	101010100	10110 0000	10 100 10000 0
32	100000	00000	111110 00000	10101010100	11100 00000	10 101 100000 0
64	1000000	000000	1111110 000000	1010101010100	11110 000000	10 110 1000000 0

7.2 C1 kodu

Slayttaki tanım (ekrandan, 01:22:50): C_1 iki parçadan oluşur. n değerinin binary karşılığı olan $B(n)$'nin dijit sayısı değerinin unary karşılığı ile $B(n)$'nin en soldaki bitinin atılmış hâli olan $\bar{B}(n)$'nin birleşimidir:

$$C_1(n) = \text{unary}(|B(n)|) \,|\, \bar{B}(n)$$

Cümle karışık görünür; hoca örneklerle açtı.

$C_1(16)$**:** $B(16) = 10000$. $\bar{B}(16) = 0000$ (en soldaki 1 atıldı). $|B(16)| = 5$. 5'in unary kodu: n 1'den başladığı için $5 - 1 = 4$ adet 1 ve bir 0 → `11110`. Arkasına `0000` → $C_1(16) = 11110|0000$. Slaytta unary kodun alternatif biçimiyle `00001|0000` yazımı da verilir (ekrandan, 01:28:40).

$C_1(12)$**:** $B(12) = 1100$ (8 + 4). En soldaki bit atılınca $\bar{B}(12) = 100$. $|B(12)| = 4$. 4'ün unary kodu: üç tane 1 ve 0 → `1110`. Sonuç $C_1(12) = 1110|100$.

$C_1(6)$**:** $B(6) = 110$, 3 dijit. 3'ün unary kodu: iki tane 1 ve 0 → `110`. $\bar{B}(6) = 10$. Sonuç $C_1(6) = 110|10$.

Bu kodda en olası sembol 1 bitle, ikinci ve üçüncü sıradakiler 3 bitle, sonrakiler 5 bitle, daha sonrakiler 7 bitle kodlanır. Hoca bunun önceki koda göre daha başarılı göründüğünü, içinde unary kod kullanıldığını söyledi.

Kod uzunluğu (ekrandan, 01:28:40):

$$|C_1(n)| = 2\lfloor \log_2 n \rfloor + 1 \text{ bit}$$

Örnek $n = 32$: $\log_2 32 = 5$; $2 \cdot 5 = 10$; $10 + 1 = 11$ bit. Tabloda $C_1(32) = 111110|00000$ gerçekten 11 bittir. Hoca bunun pratik faydasını şöyle anlattı: "Elinde 120 tane sembol var" denirse tabloda 120'nin düştüğü yere bakılıp en uzun kodun kaç bit olacağı hemen söylenebilir.

7.3 C2 kodu

C_2 tek bölümden oluşur ve C1 kodunun yeniden düzenlenmesiyle elde edilir (ekrandan, 01:32:45): C1'in birinci bölümündeki her bit teker teker alınırken her bitin arkasına ikinci bölümden bir bit yazılır.

$C_2(16)$**:** $C_1(16) = 11110|0000$. Birinci bölümden `1`, ikinciden `0`; birinciden `1`, ikinciden `0`; birinciden `1`, ikinciden `0`; birinciden `1`, ikinciden `0`; birinciden son bit `0` → $C_2(16) = 101010100$.

C2'yi bulmak için mutlaka önce C1 bulunmalı ve ardından ekstra bir işlem yapılmalıdır. C2'nin bit uzunluğu C1'inkiyle aynıdır, çünkü yalnız bitlerin sırası değişir.

7.4 C3 kodu

C_3 yine iki bölümden oluşur (ekrandan, 01:35:50):

- n'nin binary gösteriliminin dijit sayısı alınır.
- $|B(n)|$'nin değeri C2 kodu içinde bulunur ve arkasına $\bar{B}(n)$ eklenir.

$$C_3(n) = C_2(|B(n)|)\,|\,\bar{B}(n)$$

Hoca bunu “dıdının dıdısı” diye niteledi: C2 için C1, C3 için C2 bilinmelidir.

$C_3(15)$**:** $B(15) = 1111$, $|B(15)| = 4$. C2 tablosunda 4’ün kodu `10100`. $\bar{B}(15) = 111$. Sonuç $C_3(15) = 10100|111$.

$C_3(10)$**:** $B(10) = 1010$ (8 + 2). Dijit sayısı 4. $C_2(4) = 10100$. $\bar{B}(10) = 010$. Sonuç $C_3(10) = 10100|010$. Hoca önce yanlış yazdığını fark edip düzeltti; slayttaki tablo `10100|010`’dur.

Kod uzunluğu (ekrandan, 01:35:50):

$$|C_3(n)| = 1 + \lfloor \log_2 n \rfloor + 2\lfloor \log_2(1 + \lfloor \log_2 n \rfloor) \rfloor \text{ bit}$$

$n = 15$: $\log_2 16 = 4$ olduğuna göre $\log_2 15$ 3 küsurdur, alt tam değeri 3. $1 + 3 = 4$; $\log_2 4 = 2$; $2 \cdot 2 = 4$. Toplam $1 + 3 + 4 = 8$ bit. $C_3(15) = 10100111$ gerçekten 8 bittir.

7.5 C4 kodu

C_4 ötekilerden farklı çıkarılır ve birden fazla bölümden oluşur: tek bölümlü, iki, üç, dört bölümlü kodlar vardır; sayı büyüdükçe bölüm sayısı artar. Slayttaki adımlar (ekrandan, 01:40:00):

- İlk olarak hesaplanacak n’nin binary gösterilimi yazılır.
- Sol tarafına $|B(n)| - 1$ değerinin binary gösterilimi yazılır.
- İşlem, $|B(n)| - 1$ değeri iki dijit kalıncaya kadar devam ettirilir.
- En son adımda sağ tarafa 0 yazılır.

$C_4(16)$**:**

```
B(16)          = 10000                       -> |10000
|B(16)| - 1    = 5 - 1 = 4  -> B(4) = 100    -> |100|10000
|B(4)| - 1     = 3 - 1 = 2  -> B(2) = 10     -> 10|100|10000   (iki dijit, dur)
sağa 0                                        -> 10|100|10000|0
```

$C_4(64)$ (ekrandan, 01:42:45):

```
B(64)          = 1000000 (7 dijit)           -> |1000000
7 - 1 = 6      -> B(6) = 110 (3 dijit)        -> |110|1000000
3 - 1 = 2      -> B(2) = 10 (2 dijit, dur)    -> 10|110|1000000
sağa 0                                        -> 10|110|1000000|0
```

Hoca bu örnekte önce bir yerde 6 yerine 12 yazdı, sonra 3’ün binary’sini (11) ekleyip fazladan bir adım attı ve sıfırı en sola koydu; tabloyla karşılaştırınca hatayı buldu ve düzeltti. Doğru sonuç tablodaki `10|110|1000000|0`’dır.

> **Ek bilgi (derste anlatılmadı):** Slayttaki C1-C4 tablosu büyük olasılıkla D. Salomon’un veri sıkıştırma kitabındaki değişken uzunluklu kodlar tablosudur. C1 Elias gamma kodunun, C4 ise Elias omega kodunun bir yazımıdır; C3 Elias delta koduna benzer bir yapıdadır.

7.6 Kodların karşılaştırılması

Bütün kodlar slayttaki tabloyla python ile karşılaştırıldı ve tuttu. C_1’i doğrudan, C_2’yi C1’den, C_3’ü C2’den elde ederiz; C_4 ise ötekilerden tamamen bağımsızdır. Hoca kitaplarda çok daha fazla kodlama olduğunu, hepsini anlatmanın haftalar süreceğini, en popüler ve vaktiyle çok kullanılmış olanları seçtiğini söyledi.

8. Golomb Kodu

8.1 Adımlar

Golomb kodu, unary kodlama yönteminin özellikleri de kullanılarak oluşturulan basit bir prefix koddur (ekrandan, 01:50:00). Slayttaki adımlar:

- Kodları oluşturulacak alfabedeki karakterler kullanım frekansları ya da olasılıkları büyükten küçüğe doğru sıralanır.
- x olarak adlandırılan karakterlere 0'dan başlamak üzere ardışık değerler atanır (sıfırıncı, birinci, ikinci, ...).
- İdeali 2'nin kuvveti olacak bir M değeri seçilir (2, 4, 8, 16 gibi).
- Her x değeri için x_M ve x_L değerleri hesaplanır.
- Golomb kodu, x_M sayısının unary kod değeri ile x_L sayısının binary değeri birleştirilerek elde edilir.

$$x_M = \left\lfloor \frac{x}{M} \right\rfloor, \qquad x_L = x \bmod M, \qquad x = x_M \cdot M + x_L$$

8.2 Örnek: $M = 4$

Hoca dokuz karakter seçti: Türkçede en çok kullanılanlardan `a`, `e`, sonra `k`, `l`, `m`, `n`, `i`, `y` ve son harf olarak kendi adının baş harfi `b`. Karakterler olasılıklarına göre büyükten küçüğe sıralanmış kabul edildi ve 0'dan 8'e kadar değerler verildi.

x_M hesabı: $0/4$, $1/4$, $2/4$, $3/4$ sıfır küsurdur, alt tam değeri 0. $4/4 = 1$; $5/4$, $6/4$, $7/4$ bir küsurdur, alt tam değeri 1. $8/4 = 2$.

x_L hesabı: 0, 1, 2, 3'ün 4'e göre modu kendileridir; 4'ün modu 0, 5'in 1, 6'nın 2, 7'nin 3, 8'in 0.

x_M'nin unary kodu (n 0'dan başladığı için n adet 1 ve bir 0): $x_M = 0 \to$ `0`, $x_M = 1 \to$ `10`, $x_M = 2 \to$ `110`. x_L iki bitlik binary ile yazılır.

karakter	x	x_M	x_L	unary x_M	binary x_L	Golomb
a	0	0	0	`0`	`00`	`000`
e	1	0	1	`0`	`01`	`001`
k	2	0	2	`0`	`10`	`010`
l	3	0	3	`0`	`11`	`011`
m	4	1	0	`10`	`00`	`1000`
n	5	1	1	`10`	`01`	`1001`
i	6	1	2	`10`	`10`	`1010`
y	7	1	3	`10`	`11`	`1011`
b	8	2	0	`110`	`00`	`11000`

$M = 4$ olduğu için x_M dörder dörder aynı kalır: 4 tane 0, 4 tane 1, 4 tane 2 diye gider.

Kontrol: `000` ile başlayan başka kod yok; `001` ile başlayan yok; `010` ile başlayan yok. Mevcut bir kodla başlayan başka bir kod bulunamaz; bu da prefix koddur.

Hoca slayttaki "Unary kodlar, ..." ifadesinin "Golomb kodu" olması gerektiğini söyleyip düzeltti ve slaytları hafta sonu hazırladığı için o gün üç hata gördüğünü belirtti.

> **Ek bilgi (derste anlatılmadı):** M 2'nin kuvveti seçildiğinde Golomb kodu büyük olasılıkla Rice kodu adıyla da anılır. Bu durumda x_L her zaman tam $\log_2 M$ bitle yazılır, bölme ve mod işlemleri bit kaydırma ve maskelemeye dönüşür.

8.3 Ne zaman uygun?

İlk dört karakter aynı uzunlukta, 3 bitle kodlanır. `a`'nın olasılığı 0,7, `e`'ninki 0,3 olsaydı aralarında büyük fark olduğu hâlde ikisini de 3 bitle kodlamış olurduk; böyle bir kaynakta bu

kodlama uygun değildir ve artıklık yüksek olur. Gerçekte `e`'nin olasılığı bu kadar düşük değildir, `a`'ya yakındır; `k` da sık kullanılır. Olasılıklar birbirine yakınsa onları eşit bitte kodlamak artıklığı fazla artırmaz. Olasılıklar arasında çok büyük farklar varken aynı bit uzunluğu kullanılıyorsa doğru seçim yapılmamış demektir.

9. Hazır Kodlardan Yöntem Ürünü Kodlara

Hoca bu bölümü şöyle bağladı: Golomb, unary, C1-C4 ve binary sayılardan oluşan prefix kodlar görüldü. Buraya kadar olanların hepsi **önceden hazırlanmış, tablo hâlindeki** kodlardır ve elimizdeki verinin içeriğinden bağımsızdır. Bundan sonra görülecek kodlamalar ise birer yöntemdir: kodlar elimizdeki verinin içeriğine bağlı olarak oluşturulur ve hepsi prefix özelliğine sahiptir.

10. Shannon-Fano Kodlama

10.1 Yeri ve önemi

Shannon-Fano, Shannon ve Fano'nun adıyla anılan bir kodlama yöntemidir. Hayatımızda çok yer alan Huffman kodlaması bu yöntem baz alınarak görülecektir. Hoca Huffman'ın günlük hayattaki yerini şöyle örnekledi: İndirdiğimiz resimleri çoğunlukla JPEG formatında saklıyoruz, WinZip ve WinRAR kullanıyoruz; bunların hepsinin içinde ara adımda Huffman kodlaması kullanılır. Huffman Shannon-Fano'ya göre biraz daha çetrefilli olduğu için önce Shannon-Fano anlatıldı.

10.2 Adımlar

Slayttaki adımlar (ekrandan, 02:10:00):

- Değişken uzunluklu bir kodlama yöntemidir.
- Olasılık veya kullanım sıklıkları bilinen n elemanlı bir küme ile başlanır. Küme alfabedir; elemanları karakter ya da piksel olabilir. Frekans (kaç adet geçtiği) ya da olasılık kullanılabilir; ikisi de aynı kapıya çıkar.
- Semboller azalan sırada olasılıklarına ya da kullanım sıklıklarına göre sıralanır.
- Semboller kümesi aynı veya yakın olasılık değerlerine göre iki kümeye ayrılır. Tam ikiye bölünemeyebilir; iki küme arasında hafif farklar olabilir.
- Bir küme içindeki sembollerin hepsi 1 değerini alırken diğer kümedeki semboller 0 değerini alır.
- Her alt küme kendi içinde iki eleman kalıncaya kadar bölünmeye devam edilir.
- Semboller aşağıdan yukarı doğru okunur.
- Sembollerin olasılık değerleri 2'nin negatif kuvvetleri cinsinden olduğu zaman en iyi sonucu verir.

Hoca bölmeyi binary search'e benzetti: 100'lük bir şey ortadan bölünür, sonra yarısı yine bölünür ve bir durma noktasına varılır. Burada da her alt küme iki eleman kalıncaya kadar bölünür. Atanan 1 ve 0'lar sembollerin kodlarının bitleri olur. Bölmeler bir ağaç yapısı oluşturur; okuma yapraklardan köke ya da kökten yapraklara doğru yapılır.

Yöntemin en ideal olduğu, yani ortalama mesaj uzunluğunun entropiye yaklaştığı durum, olasılıkların $1/2$, $1/4$, $1/8$, $1/16$, $1/32$, $1/64$ gibi 2'nin negatif kuvvetleri olduğu durumdur. Gerçek dünyada hiçbir olay olasılıklarını böyle vermez; bu teoride kalır.

> **Ders sonrası not:** Hoca 02:11:35'te sıralamayı "en küçükten en büyüğe doğru" diye söyledi; slayta ve örneklere göre semboller azalan sırada, büyükten küçüğe sıralanır. Altyazıda 02:14:02'de "1 bölü 10" diye geçen değer 2'nin kuvvetleri dizisinde $1/16$ olmalıdır.

11. Shannon-Fano Örneği, Kod Çözme ve Değerlendirme

11.1 Bölmeler

Hoca sembollere sonradan harf verdi: `a`, `e`, `k`, `m`, `l`, `n` ve olasılığı çok küçük olduğu için `z`. Olasılıklar (ekrandan, 02:20:00):

sembol	olasılık
a	0,25
e	0,20
k	0,15
m	0,15
l	0,10
n	0,10
z	0,05

Olasılıklar toplamı 1'dir. Tam ortadan bölmek 0,5/0,5 verirdi, ama 0,5 verecek bir bölme yoktur. İlk iki sembolde bölünürse üst taraf $0{,}25+0{,}20 = 0{,}45$, alt taraf $0{,}55$; ilk üç sembolde bölünürse $0{,}60/0{,}40$. 0,45/0,55 daha dengelidir, bu yüzden ilk bölme oradan yapılır. Üst kümeye 1, alt kümeye 0 verilir. Hoca her bölmede aynı mantığı izlemek gerektiğini vurguladı: ilk kümeye 1, sonrakine 0.

Bölmeler sırasıyla:

1. $\{0{,}25; 0{,}20\} \to 1$, $\{0{,}15; 0{,}15; 0{,}10; 0{,}10; 0{,}05\} \to 0$.
2. Üst küme: $0{,}25 \to 1$, $0{,}20 \to 0$. Artık her birinde bir eleman var.
3. Alt küme: $0{,}15 + 0{,}15 = 0{,}30$ ile $0{,}10 + 0{,}10 + 0{,}05 = 0{,}25$; 30'a 25 diye bölünür. Üst kısım 1, alt kısım 0.
4. $\{0{,}15; 0{,}15\}$: tam ortadan bölünür; 1 ve 0.
5. $\{0{,}10; 0{,}10; 0{,}05\}$: $0{,}10/0{,}15$ diye bölünür; 1 ve 0.
6. $\{0{,}10; 0{,}05\}$: yaklaşık ikiye bölünür; 1 ve 0.

Ağaçta en sol kök, sembollerin bulunduğu uçlar yapraklardır.

11.2 Kodların okunması

Her sembolün kodu, onun satırındaki bitler sırayla okunarak bulunur (ekrandan, 02:20:00 ve 02:22:40):

sembol	olasılık	kod
a	0,25	`11`
e	0,20	`10`
k	0,15	`011`
m	0,15	`010`
l	0,10	`001`
n	0,10	`0001`
z	0,05	`0000`

Kodlar değişken uzunluklu mu? Evet. Prefix mi? `11` ile başlayan başka kod yok; `10` ile başlayan yok; `011` ile, `010` ile, `001` ile, `0001` ile, `0000` ile başlayan başka kod yok. Öyleyse bu da prefix koddur.

11.3 `kale` kelimesini kodlamak

`k` → `011`, `a` → `11`, `l` → `001`, `e` → `10`. Sonuç (ekrandan, 02:22:40):

```
kale -> 011 11 001 10 -> 0111100110   (10 bit)
```

> **Hocanın vurgusu:** Geçmişte ödevlerde yapılan bir hata: Kodlar dosyaya yazılırken her 0 ve 1 için bir bayt kullanılıyor. O zaman `kale` 10 bayt olur; orijinali 4 karakter, 4 bayttır. 10 bayt yazmanın bir anlamı yoktur; dosya sıkıştırılmamış, genişletilmiş olur. Bitler bayt olarak değil bit olarak alınmalıdır: ilk 8 bit kaydırılarak (shift) bir baytın içine yerleştirilir, arkadan gelen kelimelerin bitleri de böyle eklenerek bayt bayt tutulur. Bu örnekte ikinci baytta 2 bit kullanılır; `kale` için yaklaşık 2 bayt harcanır.

11.4 Kod çözme

Sıkıştırılmış veriyi açabilmek için karşı tarafın kodları bilmesi gerekir; bunun nasıl sağlanacağı Huffman'da ayrıntılı anlatılacaktır. Şimdilik karşı tarafın kodları bildiğini varsayalım ve `0111100110`'ı çözelim:

- `0`: tek bitlik kod yok. `01`: iki bitlik böyle bir kod yok. `011`: var, `k`. 011 ile başlayan başka kod olmadığı biliniyor (prefix özelliği).
- `1`: tek bitlik kod yok. `11`: var, `a`. 11 ile başlayan başka kod olmadığı için devam etmeye gerek yok.
- `0`, `00`: bunlarla başlayan çok kod var. `001`: var, `l`.
- `1`: yok. `10`: var, `e`.

Sonuç `kale`. Orijinal enkodere sokulup sıkıştırıldı (compress), sonra dekodere sokulup açıldı (decompress). Sıkıştırma ve açmanın birlikte yapıldığı kapalı kutuya **kodek** denir. Kodlar prefix olmasaydı veri bu kadar rahat açılamazdı; belki açılırdı ama yanlış kodlar gelebilirdi, örneğin `kale` yerine `kaze` gibi anlamsız bir şey çıkabilirdi ya da bir yerden sonra hiçbir kodla eşleşmeyip takılı kalınırdı.

11.5 Entropi, ortalama mesaj uzunluğu ve artıklık

Slayttaki hesaplar (ekrandan, 02:20:00):

$$H = -\big(0{,}25\log_2 0{,}25 + 0{,}20\log_2 0{,}20 + 2(0{,}15\log_2 0{,}15) + 2(0{,}10\log_2 0{,}10) + 0{,}05\log_2 0{,}05\big) = 2{,}67 \text{ bit}$$

Ortalama mesaj uzunluğu, her sembolün olasılığı çarpı kod uzunluğunun toplamıdır:

$$S = 0{,}25\cdot 2 + 0{,}20\cdot 2 + 3\cdot 0{,}15\cdot 2 + 3\cdot 0{,}10 + 4\cdot 0{,}10 + 4\cdot 0{,}05 = 2{,}7 \text{ bit}$$

Ara adımlar (ara adım eklendi): $0{,}5 + 0{,}4 + 0{,}9 + 0{,}3 + 0{,}4 + 0{,}2 = 2{,}7$.

$$R = |2{,}70 - 2{,}67| = 0{,}03 \text{ bit}$$

Teori böyle bir sistemin ortalama 2,67 bitle kodlanabileceğini söyler; Shannon-Fano ile 2,7 bit elde edildi. Artıklık sıfıra çok yakındır, bu ideal bir sıkıştırmadır. Hoca örneği bölmeler kolay ve güzel görünsün diye özellikle böyle hazırladığını da ekledi.

> **Ders sonrası not:** Hoca 02:27:58'de S'yi hesaplarken önce iki 0,15'i ve iki 0,10'u 3 bit saydı, sonra düzeltti: 0,10'lardan biri 3 bit (`001`), diğeri 4 bittir (`0001`). Slayttaki formül doğrudur. H python ile 2,666 bit çıktı, 2,67'ye yuvarlanır.

11.6 Header ve doğrulama

Yapılacak her şey artık budur: Elde bir doküman vardır, bu doküman sıkıştırılır. Karşı tarafın açabilmesi için başına bir **header** bilgisi eklenir. Header'da olasılık değerleri, sembollerin kodları ya da başka bir bilgi olabilir; bunlar yeri geldikçe anlatılacaktır. Karşı taraf önce header'dan nasıl kodlandığını öğrenir, sonra sıkıştırılmış kısmı açar.

Açılan dosya orijinalle karşılaştırılır. Kayıpsız sıkıştırmada iki dosya bayt olarak karşılaştırıldığında (compare) birebir aynı çıkmalıdır. Kayıplı sıkıştırmada birebir aynılık beklenmez. Görüntüde başarı root mean square ya da signal-to-noise ratio ile ölçülür. Metinde kayıplı sıkıştırma biraz problemlidir, çünkü bit bit aynı çıkmaz; ama açıldığında içerik okunabiliyor ve bir bozulma yoksa bu da kabul edilebilir bir sistemdir.

> **Ders sonrası not:** 02:30:50'de altyazıda "kayıplı sıkıştırma yapmıyorsam, kayıpsız sıkıştırma yapıyorsam bu ikisi birbirinden farklı olabilir" geçiyor; hocanın hemen sonraki cümlelerine göre kastedilen, kayıpsızda iki dosyanın birebir aynı, kayıplıda farklı olabileceğidir.

11.7 Kodlar veriye bağlıdır

Shannon-Fano'da kodlar önceden hazırlanmamıştır; verinin içeriğine göre oluşturulur. Olasılıklar farklı olsaydı, örneğin ilk sembol 0,75 olsaydı, kodlar da bambaşka çıkacaktı.

12. İkinci Shannon-Fano Örneği

Hoca tahtada yeni bir örnek kurdu (ekrandan, 02:35:30). Olasılıklar:

$$0{,}4; \quad 0{,}4; \quad 0{,}1; \quad 0{,}05; \quad 0{,}05$$

Toplam 1'dir. Bölmeler:

1. Üstte 0,4, altta 0,6 kalacak şekilde bölünür. 0,8/0,2 bölmesi hiç adil olmaz. İlk eleman 1, diğerleri 0.
2. Kalan $\{0{,}4; 0{,}1; 0{,}05; 0{,}05\}$: 0,5'e 0,1 olmaz; 0,4'e 0,2 diye bölünür. 0,4 → 1, diğerleri 0.
3. $\{0{,}1; 0{,}05; 0{,}05\}$: 0,1'e 0,1 diye bölünür. 0,1 → 1, diğerleri 0.
4. $\{0{,}05; 0{,}05\}$: tam ortadan bölünür; 1 ve 0.

Kodlar: `1`, `01`, `001`, `0001`, `0000`. Kontrol: `1` ile, `01` ile, `001` ile, `0001` ile, `0000` ile başlayan başka kod yoktur; bu da prefix koddur.

Her zaman tam ortadan bölmek mümkün olmaz; bu örnekte 0,4'e 0,6 alındı, çünkü alınabilecek en yakın olasılık değerleri budur.

Hoca Huffman'ın görünüş olarak Shannon-Fano'ya benzediğini ama mantığının çok daha farklı olduğunu, çok önemli bir algoritma olduğu için birçok aşamasını derinlemesine anlatacağını söyledi.

Örnek Sorular

Soru 1 (ekrandan, 00:20:00): $\{a_1, a_2, a_3, a_4\}$ sembollerinin hepsi eşit olasılığa sahip olsun ($P_i = 0{,}25$). Entropiyi, 2 bitlik sabit kodla ortalama mesaj uzunluğunu ve artıklığı bulun. Olasılıklar $0{,}49$; $0{,}25$; $0{,}25$; $0{,}01$ olsaydı ve yine 2 bitlik kod kullanılsaydı artıklık ne olurdu?

> **Çözüm (derste verildi):** $H = -(4 \cdot 0{,}25 \log_2 0{,}25) = 2$ bit; `00 01 10 11` ile $S = 2$, $R = 0$. İkinci durumda $H = 1{,}57$ bit, $S = 2$, $R = |2 - 1{,}57| = 0{,}43$.

Soru 2 (ekrandan, 00:30:00): Input `a1 a3 a2 a1 a3 a3 a4 a2 a1 a1 a2 a2 a1 a1 a3 a1 a1 a2 a3 a1` için Code1 (`1, 01, 010, 001`) ve Code2 (`1, 01, 000, 001`) verilmiştir. `a1 a3 a1` girdisini iki kodla kodlayıp çözün. Hangi kod prefixtir? 20 sembol kaç bitle kodlanır, ortalama mesaj uzunluğu ve artıklık nedir?

> **Çözüm (derste verildi, düzeltmeyle):** Code1: `10101`; çözmede `01` mi (`a2`) `010` mu (`a3`) karar verilemez, Code1 prefix değildir. Code2: `10001` → `a1 a3 a1`, prefixtir. 20 sembol: $9 \cdot 1 + 5 \cdot 2 + 5 \cdot 3 + 1 \cdot 3 = 37$ bit (ASCII ile 160 bit). $S = 37/20 = 1{,}85$; slayta göre $H = 1{,}57$, $R = 0{,}28$. Girdideki sayımlarla ($0{,}45$; $0{,}25$; $0{,}25$; $0{,}05$) $H \approx 1{,}73$ ve $R \approx 0{,}11$ olur (python ile).

Soru 3 (derste sözlü, 00:33:43): Integer sayıların binary gösterilimi prefix kod özelliğine uyar mı?

> **Çözüm (derste verildi):** Uymaz. `0`, `1`, `10`, `11`, `100`, … dizisinde `1` kodu `10`, `11`, `100` gibi kodların ön ekidir; `10` da `100` ve `101`'in ön ekidir. Hepsi aynı bit sayısıyla yazılırsa prefix olur ama kod sabit uzunluklu olur ve artıklık yüksek kalır.

Soru 4 (ekrandan, 00:44:40 ve 00:47:20): $i \geq 1$ olmak şartıyla i. kodun binary değerinin başına $\lfloor \log_2 i \rfloor$ adet sıfır eklenir. $i = 5$ ve $i = 15$ için kodu yazın. i kodunun toplam bit uzunluğu nedir? $i = 16$ ve $i = 6$ için uzunluğu bulun.

> **Çözüm (derste verildi):** $5 \to$ `00101`, $15 \to$ `0001111`. Uzunluk $1 + 2\lfloor \log_2 i \rfloor$: $i = 16$ için $1 + 2 \cdot 4 = 9$ bit, $i = 6$ için $1 + 2 \cdot 2 = 5$ bit.

Soru 5 (ekrandan, 01:10:00 ve 01:14:40): 680 adet unary kodu nasıl elde edebiliriz? Triplet $(3, 2, 9)$ için kod sözcüklerini ve aralıklarını çıkarın. Start, step ve stop değerlerine bağlı olarak kaç adet unary kod oluşturulur?

> **Çözüm (derste verildi):** $a = 3 + 2n$: $3, 5, 7, 9$. Kod sözcükleri `0xxx` (8 kod, 0-7), `10xxxxx` (32 kod, 8-39), `110xxxxxxx` (128 kod, 40-167), `111xxxxxxxxx` (512 kod, 168-679); $a =$ stop olduğu için son gruptaki 0 düşer. Kod sayısı $\frac{2^{9+2} - 2^3}{2^2 - 1} = \frac{2048 - 8}{3} = 680$; stop'a bağlı olarak üstel artar.

Soru 6 (ekrandan, 01:22:50 ve 01:28:40): $C_1(16)$, $C_1(12)$ ve $C_1(6)$ kodlarını bulun. $C_1(32)$ kaç bittir?

> **Çözüm (derste verildi):** $C_1(16) = 11110|0000$, $C_1(12) = 1110|100$, $C_1(6) = 110|10$. Uzunluk $2\lfloor \log_2 n \rfloor + 1$: $n = 32$ için $2 \cdot 5 + 1 = 11$ bit.

Soru 7 (ekrandan, 01:32:45, 01:35:50, 01:40:00 ve 01:42:45): $C_2(16)$, $C_3(15)$, $C_3(10)$, $C_4(16)$ ve $C_4(64)$ kodlarını bulun. $C_3(15)$'in uzunluğunu formülle doğrulayın.

> **Çözüm (derste verildi):** $C_2(16) = 101010100$ (C1'in iki bölümünün bitleri sırayla). $C_3(15) = C_2(4)|\bar{B}(15) = 10100|111$, $C_3(10) = 10100|010$. Uzunluk $1 + \lfloor \log_2 15 \rfloor + 2\lfloor \log_2(1 + 3) \rfloor = 1 + 3 + 4 = 8$ bit. $C_4(16) = 10|100|10000|0$, $C_4(64) = 10|110|1000000|0$. Bütün kodlar python ile slayttaki tabloyla doğrulandı.

Soru 8 (ekrandan, 01:50:00 ve 01:52:30): Olasılıklarına göre sıralanmış dokuz karakter (`a e k l m n i y b`) için $M = 4$ ile Golomb kodlarını çıkarın. Kodlar prefix midir?

> **Çözüm (derste verildi):** $x = 0..8$; $x_M = \lfloor x/4 \rfloor$: `0 0 0 0 1 1 1 1 2`; $x_L = x \bmod 4$: `0 1 2 3 0 1 2 3 0`. Kodlar: `000`, `001`, `010`, `011`, `1000`, `1001`, `1010`, `1011`, `11000`. Hiçbir kod başka bir kodun ön eki olmadığı için prefixtir.

Soru 9 (ekrandan, 02:20:00 ve 02:22:40): Olasılıkları $0{,}25$; $0{,}20$; $0{,}15$; $0{,}15$; $0{,}10$; $0{,}10$; $0{,}05$ olan semboller (`a e k m l n z`) için Shannon-Fano kodlarını bulun. Entropi, ortalama mesaj uzunluğu ve artıklığı hesaplayın. `kale` kelimesini kodlayıp çözün.

> **Çözüm (derste verildi):** Kodlar `a=11`, `e=10`, `k=011`, `m=010`, `l=001`, `n=0001`, `z=0000`. $H = 2{,}67$ bit, $S = 2{,}7$ bit, $R = 0{,}03$ bit. `kale` → `0111100110` (10 bit); çözmede `011` → k, `11` → a, `001` → l, `10` → e.

Soru 10 (ekrandan, 02:35:30): Olasılıkları $0{,}4$; $0{,}4$; $0{,}1$; $0{,}05$; $0{,}05$ olan sembollere Shannon-Fano uygulayın.

> **Çözüm (derste verildi):** Bölmeler $0{,}4/0{,}6$, $0{,}4/0{,}2$, $0{,}1/0{,}1$, $0{,}05/0{,}05$. Kodlar `1`, `01`, `001`, `0001`, `0000`; prefix koddur.
>
> **Çözüm (derste verilmedi):** Python ile: $S = 0{,}4 \cdot 1 + 0{,}4 \cdot 2 + 0{,}1 \cdot 3 + 0{,}05 \cdot 4 + 0{,}05 \cdot 4 = 1{,}9$ bit, $H \approx 1{,}82$ bit, $R \approx 0{,}08$ bit.

Dersin Toparlaması

Bu derste istatistiksel yöntemlerin temel şartı olan prefix kodlar işlendi. Sabit uzunluklu kodların olasılıklar farklıyken artıklığı büyüttüğü, bu yüzden değişken uzunluklu kodlara geçilmesi gerektiği gösterildi. Değişken uzunluklu kodun geri açılabilmesi için hiçbir kodun başka bir kodun ön eki olmaması gerekir; Code1 ve Code2 örneğinde aynı bit uzunluklarına sahip iki koddan yalnız prefix olanın sorunsuz çözüldüğü görüldü. Ardından verinin içeriğinden bağımsız, tablo hâlinde hazırlanmış prefix kodlar anlatıldı: başa $\lfloor \log_2 i \rfloor$ sıfır ekleyen kod, unary kod ve start-step-stop tripleti, $B(n)$, $|B(n)|$ ve $\bar{B}(n)$ üzerinden kurulan C1, C2, C3, C4 kodları ve unary ile binary kısmı birleştiren Golomb kodu. Bu kodların uzunluk formülleri sayesinde tabloyu kurmadan bir kodun kaç bit olacağı bulunabilir. Ancak hazır kodlar veriye bakmadan kullanıldığında artıklık yüksek kalabilir.

Son bölümde kodları verinin içeriğine göre üreten ilk yöntem olan Shannon-Fano kodlama görüldü: semboller azalan sırada dizilir, olasılık toplamları birbirine en yakın iki kümeye bölünür, kümelere 1 ve 0 verilir ve bu işlem tekrarlanır. Örnekte artıklık 0,03 bit çıktı; `kale` kelimesi

10 bitle kodlanıp prefix özelliği sayesinde sorunsuz çözüldü. Hoca bitlerin dosyaya bayt olarak değil bit olarak yazılması gerektiğini ve alıcının kodları bilmesi için header kullanıldığını vurguladı. Sonraki derste, Shannon-Fano'ya görünüşte benzeyen ama mantığı çok farklı olan Huffman kodlamasıyla devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.